Bayesian multimodeling: Variational inference and diffusion

MIPT

2026

Recap

Bayesian inference

- The posterior combines prior knowledge and observed data.
- The evidence is central to Bayesian model comparison.

Approximate inference

- **Laplace approximation:** approximate the posterior locally by a Gaussian around its mode.
- **Variational inference:** optimize a tractable distribution by maximizing the ELBO, a lower bound on the log evidence.
- **Expectation propagation:** approximate the posterior through iterative moment matching of local factors.
- **Probabilistic backpropagation:** combine moment propagation and backpropagation to perform approximate inference in neural networks.
- **MCMC:** construct samples from the posterior to estimate its properties.

Key trade-off

- Computational efficiency vs. approximation accuracy.
- Flexibility of the approximation vs. tractability.
- Quality of posterior approximation vs. quality of evidence estimation.

Variational inference: beyond standard assumptions

- **Does the variational distribution have to be Gaussian?**
  - No. Many other distribution families are possible.
- **Can variational inference be interpreted through information theory and compression?**
  - Yes. The bits-back argument connects the ELBO to the description length of data under a probabilistic model.
- **Does it need to have an analytically tractable density?**
  - No. Implicit distributions can also be used.
- **Is it always about KL minimization?**
  - No. We can replace the standard KL divergence with a Rényi divergence.
- **Does it have to belong to a parametric family?**
  - No. Nonparametric variational approximations are also possible.

Gradient of an expectation

Consider an expectation with respect to a parameterized distribution:

$$J(\theta) = \mathbb{E}_{x \sim q_\theta(x)}[f_\theta(x)].$$

Its gradient can be written as a sum of two terms:

$$\begin{aligned}\nabla_\theta J(\theta) = &\mathbb{E}_{q_\theta(x)}[\nabla_\theta f_\theta(x)] \\ &+ \mathbb{E}_{q_\theta(x)}\left[f_\theta(x) \nabla_\theta \log q_\theta(x)\right].\end{aligned}$$

The first term differentiates the function, while the second accounts for changes in the distribution.
The score $\nabla_\theta \log q_\theta(x)$ may be difficult or impossible to compute.

Reparameterization trick

Reparameterization idea:
Represent the model parameters as a differentiable transformation of parameter-independent noise:

$$\varepsilon \sim p(\varepsilon), \qquad \mathbf{w} = S_{\boldsymbol{\theta}}(\varepsilon).$$

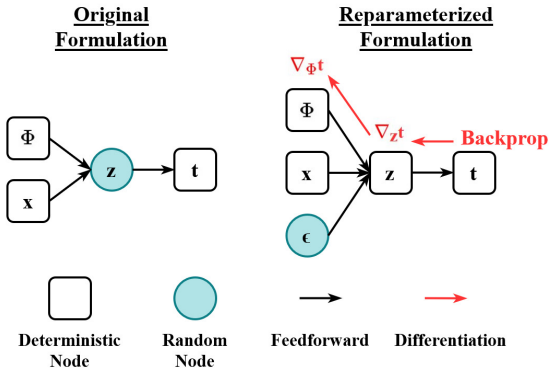
Then the gradient of an expectation can be computed as:

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{q_\theta(\mathbf{w})}[f(\mathbf{w})] = \mathbb{E}_{p(\varepsilon)}\left[\nabla_{\boldsymbol{\theta}} f(S_{\boldsymbol{\theta}}(\varepsilon))\right].$$

Example:

$$w \sim \mathcal{N}(\mu, \sigma^2), \quad w = \mu + \sigma\varepsilon, \quad \varepsilon \sim \mathcal{N}(0,1).$$

Challenge: finding an explicit differentiable transformation can be difficult.



Source: Wikipedia

Normalizing Flows

Starting from a simple distribution over model parameters,

$$\mathbf{w}_0 \sim q_0,$$

we construct a more flexible variational distribution through a sequence of invertible transformations:

$$\mathbf{w}_k = g_k(\mathbf{w}_{k-1}), \quad k = 1, \ldots, K.$$

The resulting density is obtained using the change-of-variables formula:

$$\log q_K(\mathbf{w}_K) = \log q_0(\mathbf{w}_0) - \sum_{k=1}^{K} \log \left| \det \frac{\partial g_k}{\partial \mathbf{w}_{k-1}} \right|.$$

Example: Planar flow

$$g(\mathbf{w}) = \mathbf{w} + \mathbf{u} \tanh(\mathbf{v}^{\mathsf{T}} \mathbf{w} + b).$$

- Flexible variational distributions.
- Tractable density evaluation through Jacobian determinants.

Reparameterization: a limitation

- Standard reparameterization requires an explicit differentiable transformation of parameter-independent noise.
- What if the variational distribution is known only through its density or cumulative distribution function?
- Can we still compute gradients through samples without explicitly constructing the transformation?

Implicit reparameterization trick

Suppose the variational distribution over model parameters is implicitly defined by

$$F_{\boldsymbol{\theta}}(\mathbf{w}) = \boldsymbol{\varepsilon},$$

where $\boldsymbol{\varepsilon}$ has a parameter-independent distribution.
Differentiating with respect to $\boldsymbol{\theta}$:

$$\nabla_{\boldsymbol{\theta}} F_{\boldsymbol{\theta}}(\mathbf{w}) + \nabla_{\mathbf{w}} F_{\boldsymbol{\theta}}(\mathbf{w}) \nabla_{\boldsymbol{\theta}} \mathbf{w} = 0.$$

Therefore:

$$\nabla_{\boldsymbol{\theta}} \mathbf{w} = -\left(\nabla_{\mathbf{w}} F_{\boldsymbol{\theta}}(\mathbf{w})\right)^{-1} \nabla_{\boldsymbol{\theta}} F_{\boldsymbol{\theta}}(\mathbf{w}).$$

The gradient of an expectation becomes:

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{q_\theta(\mathbf{w})}[f(\mathbf{w})] = \mathbb{E}_{q_\theta(\mathbf{w})}\left[\nabla_{\mathbf{w}} f(\mathbf{w}) \nabla_{\boldsymbol{\theta}} \mathbf{w}\right].$$

Key idea: compute gradients without explicitly constructing or differentiating the inverse transformation.

Table 4: Test negative log-likelihood (lower is better) for VAE on MNIST. Mean $\pm$ standard deviation over 5 runs. The von Mises-Fisher results are from [9].

Prior	Variational posterior	$D = 2$	$D = 5$	$D = 10$	$D = 20$	$D = 40$
$\mathcal{N}(0, 1)$	$\mathcal{N}(\mu, \sigma^2)$	131.1 ± 0.6	107.9 ± 0.4	92.5 ± 0.2	88.1 ± 0.2	88.1 ± 0.0
$\text{Gamma}(0.3, 0.3)$	$\text{Gamma}(\alpha, \beta)$	132.4 ± 0.3	108.0 ± 0.3	94.0 ± 0.3	90.3 ± 0.2	90.6 ± 0.2
$\text{Gamma}(10, 10)$	$\text{Gamma}(\alpha, \beta)$	135.0 ± 0.2	107.0 ± 0.2	92.3 ± 0.2	88.3 ± 0.2	88.3 ± 0.1
$\text{Uniform}(0, 1)$	$\text{Beta}(\alpha, \beta)$	128.3 ± 0.2	107.4 ± 0.2	94.1 ± 0.1	88.9 ± 0.1	88.6 ± 0.1
$\text{Beta}(10, 10)$	$\text{Beta}(\alpha, \beta)$	131.1 ± 0.4	$\mathbf{106.7} \pm 0.1$	$\mathbf{92.1} \pm 0.2$	$\mathbf{87.8} \pm 0.1$	$\mathbf{87.7} \pm 0.1$
$\text{Uniform}(-\pi, \pi)$	$\text{vonMises}(\mu, \kappa)$	$\mathbf{127.6} \pm 0.4$	107.5 ± 0.4	94.4 ± 0.5	90.9 ± 0.1	91.5 ± 0.4
$\text{vonMises}(0, 10)$	$\text{vonMises}(\mu, \kappa)$	130.7 ± 0.8	107.5 ± 0.5	92.3 ± 0.2	$\mathbf{87.8} \pm 0.2$	87.9 ± 0.3
$\text{Uniform}(S^D)$	$\text{vonMisesFisher}(\boldsymbol{\mu}, \kappa)$	132.5 ± 0.7	108.4 ± 0.1	93.2 ± 0.1	89.0 ± 0.3	90.9 ± 0.3

Bits-Back coding: Alice and Bob

Alice wants to transmit dataset $\mathfrak{D}$ to Bob. They agree on a model $p(\mathfrak{D}|\mathbf{w})$, a prior $p(\mathbf{w})$, and a learning algorithm.

1. Alice runs the learning algorithm on $\mathfrak{D}$ to obtain a variational posterior $q(\mathbf{w})$.
2. She samples $\mathbf{w} \sim q(\mathbf{w})$ using random bits and sends $\mathbf{w}$ to Bob, encoded using the prior $p(\mathbf{w})$.
3. Alice encodes and sends $\mathfrak{D}$ using $p(\mathfrak{D}|\mathbf{w})$.
4. Bob decodes $\mathbf{w}$ and $\mathfrak{D}$. He runs the same learning algorithm to reconstruct $q(\mathbf{w})$ and recovers the random bits Alice used to sample $\mathbf{w}$.

Bits-back: The random bits used to sample $\mathbf{w}$ are effectively returned to Alice's communication budget. Their expected number is $\mathsf{S}(q)$.
Thus, the expected net coding cost is

$$\begin{aligned} L(\mathfrak{D}) &= \mathbb{E}_{q(\mathbf{w})}[-\log p(\mathfrak{D}|\mathbf{w}) - \log p(\mathbf{w})] - \mathsf{S}(q) \\ &= \mathbb{E}_{q(\mathbf{w})}[-\log p(\mathfrak{D}|\mathbf{w})] + \mathrm{KL}(q(\mathbf{w})\|p(\mathbf{w})) \\ &= -\mathrm{ELBO}. \end{aligned}$$

Rényi Divergence Variational Inference

Rényi's α-divergence

$$D_\alpha[p\|q] = \frac{1}{\alpha - 1} \log \int p(\boldsymbol{\theta})^\alpha q(\boldsymbol{\theta})^{1-\alpha} \, d\boldsymbol{\theta}.$$

- D_α is continuous and non-decreasing in α wherever it is finite.
- As $\alpha \to 1$:

$$D_\alpha[p\|q] \to \mathrm{KL}(p\|q).$$

- Skew symmetry, for $\alpha \notin \{0, 1\}$:

$$D_\alpha[p\|q] = \frac{\alpha}{1 - \alpha} D_{1-\alpha}[q\|p].$$

  In particular, $D_\alpha[p\|q] \leq 0$ for $\alpha < 0$.

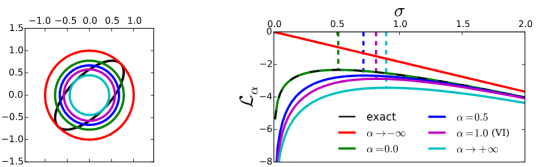
See also talks by Kseniia Petrushina and Galina Boeva, 2023.

Different divergence functions

α	Definition	Notes
$\alpha \to 1$	$\int p(\boldsymbol{\theta}) \log \frac{p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})} d\boldsymbol{\theta}$	*Kullback-Leibler (KL) divergence*, used in VI ($\mathrm{KL}[q\|\|p]$) and EP ($\mathrm{KL}[p\|\|q]$)
$\alpha = 0.5$	$-2\log(1 - \mathrm{Hel}^2[p\|\|q])$	function of the square *Hellinger distance*
$\alpha \to 0$	$-\log \int_{p(\boldsymbol{\theta})>0} q(\boldsymbol{\theta}) d\boldsymbol{\theta}$	zero when $\mathrm{supp}(q) \subseteq \mathrm{supp}(p)$ (not a divergence)
$\alpha = 2$	$-\log(1 - \chi^2[p\|\|q])$	proportional to the χ^2-divergence
$\alpha \to +\infty$	$\log \max_{\boldsymbol{\theta} \in \Theta} \frac{p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})}$	*worst-case regret* in *minimum description length principle* [24]

- When $\alpha = 0$, we get an approximate Evidence (like in IWAE).
- When $\alpha = 0.5$, we get ELBO.
- When $\alpha \to \infty$ we get mode-seeking (also called zero-forcing) optimization.
- When $\alpha \to -\infty$ we get mass-preserving optimization.

Gaussian example



(a) Approximated posterior. (b) Hyper-parameter optimisation.

Figure 1: Mean-Field approximation for Bayesian linear regression. In this case $\varphi = \sigma$ the observation noise variance. The bound is tight as $\sigma \to +\infty$, biasing the VI solution to large σ values.

Note, $\alpha \to \infty$ works similar to MAP, but still can give some non-degenerate probabilistic estimations.

MCMC and variational inference

MCMC idea: Sample from the simple distribution and accept samples, if the ratio is greater than some threshold:

$$\alpha = \min\left(1, \frac{p(\mathbf{w}'|\mathfrak{D})r(\mathbf{w}^{\tau-1}|\mathbf{w}')}{p(\mathbf{w}^{\tau-1}|\mathfrak{D})r(\mathbf{w}'|\mathbf{w}^{\tau-1})}\right),$$

where $\boldsymbol{w}^\tau$ is set based on the previous sample:

$$\boldsymbol{w}^\tau = T(\boldsymbol{w}^{\tau-1}).$$

Salimans et al., 2014: let's interperete the sequence of some operator T application as a variational optimization:

$$T^1 \circ \dots T^\eta(\boldsymbol{w}) \to p(\boldsymbol{w}^\tau|\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{h}).$$

Maclaurin et. al, 2015: use gradient descent as such operator. Do not reject samples at all.

Gradient descent for evidence estimation

Consider posterior probability maximization:

$$L = -\log\, p(\mathfrak{D}, \boldsymbol{w}|\boldsymbol{h}) = -\sum_{\mathfrak{D}\in\mathfrak{D}} \log\, p(\mathfrak{D}|\boldsymbol{w}, \boldsymbol{h}) - \log\; p(\boldsymbol{w}|\boldsymbol{h})$$

Optimize neural network in a multi-start regime with r initial parameter values $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_r$ using (stochastic) gradient descent:

$$\boldsymbol{w}' = T(\boldsymbol{w}).$$

The parameter vectors $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_r$ are from some latent distribution $q(\boldsymbol{w})$.

Entropy

We can rewrite variational inference using differential entropy term:

$$\log p(\mathfrak{D}|\boldsymbol{f}) \geq \int_{\boldsymbol{w}} q(\boldsymbol{w}) \log \frac{p(\mathfrak{D}, \boldsymbol{w}|\boldsymbol{h})}{q(\boldsymbol{w})} d\boldsymbol{w} =$$

$$\mathsf{E}_{q(\boldsymbol{w})}[\log p(\mathfrak{D}, \boldsymbol{w}|\boldsymbol{h})] + \mathsf{S}(q(\boldsymbol{w})),$$

where $\mathsf{S}(q(\boldsymbol{w}))$ is a differential entropy:

$$\mathsf{S}(q(\boldsymbol{w})) = -\int_{\boldsymbol{w}} q(\boldsymbol{w}) \log q(\boldsymbol{w}) d\boldsymbol{w}.$$

Gradient descent for evidence estimation

Statement

Let L be twice differentiable with a Lipschitz-continuous gradient, and optimization operator be a bijection. Then entropy difference for two steps is:

$$\mathsf{S}(q'(\boldsymbol{w})) - \mathsf{S}(q(\boldsymbol{w})) \simeq \frac{1}{r}\sum_{g=1}^{r}\big(-\beta\mathsf{Tr}[\boldsymbol{H}(\boldsymbol{w}'^g)] - \beta^2\mathsf{Tr}[\boldsymbol{H}(\boldsymbol{w}'^g)\boldsymbol{H}(\boldsymbol{w}'^g)]\big).$$

Final estimation for the τ optimization step:

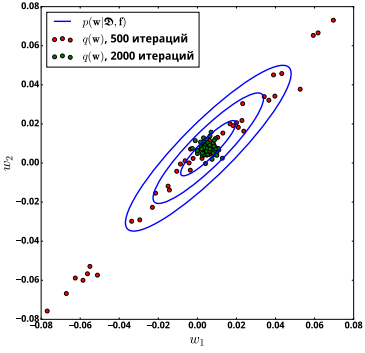
$$\log\hat{p}(\boldsymbol{Y}|\mathfrak{D},\boldsymbol{h}) \sim -\frac{1}{r}\sum_{g=1}^{r}L(\boldsymbol{w}_\tau^g,\mathfrak{D},\boldsymbol{Y}) + \mathsf{S}(q^0(\boldsymbol{w}))+$$

$$+\frac{1}{r}\sum_{b=1}^{\tau}\sum_{g=1}^{r}\big(-\beta\mathsf{Tr}[\boldsymbol{H}(\boldsymbol{w}_b^g)] - \beta^2\mathsf{Tr}[\boldsymbol{H}(\boldsymbol{w}_b^g)\boldsymbol{H}(\boldsymbol{w}_b^g)]\big),$$

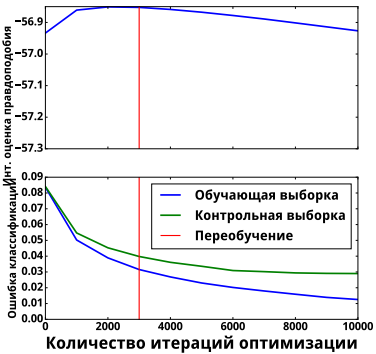
$\boldsymbol{w}_b^g$ is a parameter vector for optimization g on the step b, $\mathsf{S}(q^0(\boldsymbol{w}))$ is an initial entropy.

Overfitting, Maclaurin et. al, 2015

Gradient descent does not optimize KL-divergence $\mathrm{KL}(q(\boldsymbol{w})||p(\boldsymbol{w}|\mathfrak{D}, \boldsymbol{h}))$. Evidence estimation gets worse while optimization tends to the optimal parameter values. This can be considered as a overfitting start.



Convergence



Overfitting start

Stochastic gradient Langevin dynamics

A modification of SGD:

$$T = \boldsymbol{w} - \beta \nabla L + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \frac{\beta}{2})$$

where β changes with a number of iterations:

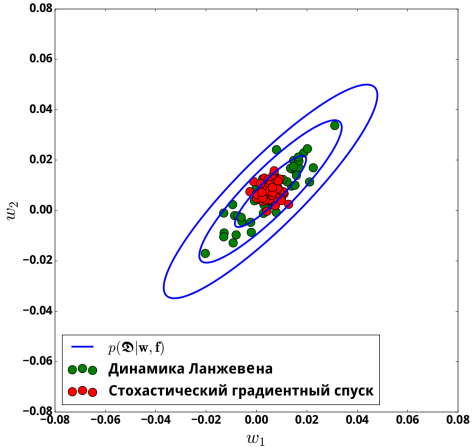
$$\sum_{\tau=1}^{\infty} \beta_\tau = \infty, \quad \sum_{\tau=1}^{\infty} \beta_\tau^2 < \infty.$$

Statement [Welling, 2011]. Distribution $q^\tau(\boldsymbol{w})$ converges to posterior distribution $p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{f})$.
Entropy adjustment:

$$\hat{\mathsf{S}}\big(q^\tau(\boldsymbol{w})\big) \geq \frac{1}{2}|\boldsymbol{w}| \log\Big(\exp\big(\frac{2\mathsf{S}(q^\tau(\boldsymbol{w}))}{|\boldsymbol{w}|}\big) + \exp\big(\frac{2\mathsf{S}(\epsilon)}{|\boldsymbol{w}|}\big)\Big).$$

SGLD vs SGD

Parameter distribution after 2000 iterations:



Stein operator and SVGD

Given a smooth probability density p and a smooth vector function $\boldsymbol{\phi}$, define the Stein operator:

$$\mathcal{A}_p \boldsymbol{\phi}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}) \boldsymbol{\phi}(\mathbf{x})^{\mathsf{T}} + \nabla_{\mathbf{x}} \boldsymbol{\phi}(\mathbf{x}).$$

Stein's identity:

$$\mathbb{E}_{\mathbf{x} \sim p} \operatorname{tr} \mathcal{A}_p \boldsymbol{\phi}(\mathbf{x}) = 0.$$

For $q \neq p$, this expectation is generally non-zero.
Consider a transformation $T(\mathbf{x}) = \mathbf{x} + \varepsilon \boldsymbol{\phi}(\mathbf{x})$:

$$\frac{\partial}{\partial \varepsilon} \mathrm{KL}(T_{\#} q \| p) \bigg|_{\varepsilon = 0} = -\mathbb{E}_{\mathbf{x} \sim q} \operatorname{tr} \mathcal{A}_p \boldsymbol{\phi}(\mathbf{x}).$$

SVGD: For a positive definite kernel K, the optimal direction in the associated RKHS is:

$$\boldsymbol{\phi}^*(\mathbf{x}') = \mathbb{E}_{\mathbf{x} \sim q} \left[K(\mathbf{x}, \mathbf{x}') \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \nabla_{\mathbf{x}} K(\mathbf{x}, \mathbf{x}') \right].$$

Stein operator: algorithm

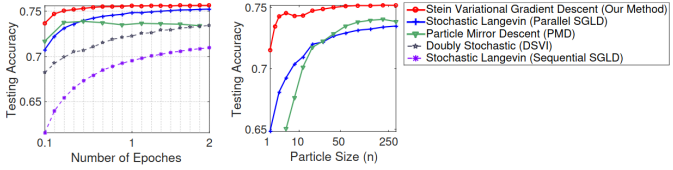
Algorithm 1 Bayesian Inference via Variational Gradient Descent

Input: A target distribution with density function $p(x)$ and a set of initial particles $\{x_i^0\}_{i=1}^n$.
Output: A set of particles $\{x_i\}_{i=1}^n$ that approximates the target distribution $p(x)$.
for iteration ℓ **do**

$$x_i^{\ell+1} \leftarrow x_i^\ell + \epsilon_\ell \hat{\phi}^*(x_i^\ell) \quad \text{where} \quad \hat{\phi}^*(x) = \frac{1}{n}\sum_{j=1}^{n}\left[k(x_j^\ell, x)\nabla_{x_j^\ell}\log p(x_j^\ell) + \nabla_{x_j^\ell} k(x_j^\ell, x)\right], \quad (8)$$

where ϵ_ℓ is the step size at the ℓ-th iteration.
end for

Stein operator: results



(a) Particle size $n = 100$ (b) Results at 3000 iteration (≈ 0.32 epoches)

	Avg. Test RMSE		Avg. Test LL		Avg. Time (Secs)	
Dataset	**PBP**	**Our Method**	**PBP**	**Our Method**	**PBP**	**Ours**
Boston	2.977 ± 0.093	$\mathbf{2.957 \pm 0.099}$	-2.579 ± 0.052	$\mathbf{-2.504 \pm 0.029}$	18	**16**
Concrete	5.506 ± 0.103	$\mathbf{5.324 \pm 0.104}$	-3.137 ± 0.021	$\mathbf{-3.082 \pm 0.018}$	33	**24**
Energy	1.734 ± 0.051	$\mathbf{1.374 \pm 0.045}$	-1.981 ± 0.028	$\mathbf{-1.767 \pm 0.024}$	25	**21**
Kin8nm	0.098 ± 0.001	$\mathbf{0.090 \pm 0.001}$	0.901 ± 0.010	$\mathbf{0.984 \pm 0.008}$	118	**41**
Naval	0.006 ± 0.000	$\mathbf{0.004 \pm 0.000}$	3.735 ± 0.004	$\mathbf{4.089 \pm 0.012}$	173	**49**
Combined	4.052 ± 0.031	$\mathbf{4.033 \pm 0.033}$	-2.819 ± 0.008	$\mathbf{-2.815 \pm 0.008}$	136	**51**
Protein	4.623 ± 0.009	$\mathbf{4.606 \pm 0.013}$	-2.950 ± 0.002	$\mathbf{-2.947 \pm 0.003}$	682	**68**
Wine	0.614 ± 0.008	$\mathbf{0.609 \pm 0.010}$	-0.931 ± 0.014	$\mathbf{-0.925 \pm 0.014}$	26	**22**
Yacht	$\mathbf{0.778 \pm 0.042}$	0.864 ± 0.052	$\mathbf{-1.211 \pm 0.044}$	-1.225 ± 0.042	25	25
Year	$8.733 \pm$ NA	$\mathbf{8.684 \pm}$ NA	$-3.586 \pm$ NA	$\mathbf{-3.580 \pm}$ NA	7777	**684**

From SGLD to diffusion

Recall the SGLD update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\beta_t}{2}\nabla_{\mathbf{w}} L(\mathbf{w}_t) + \sqrt{\beta_t}\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, I).$$

This can be interpreted as a discretization of a stochastic differential equation:

$$d\mathbf{w} = \underbrace{-\frac{1}{2}\nabla_{\mathbf{w}} L(\mathbf{w})\, dt}_{\text{Drift}} + \underbrace{d\mathbf{B}_t}_{\text{Diffusion}},$$

where $\mathbf{B}_t$ is standard Brownian motion.

- **Drift:** deterministic motion towards regions of higher posterior probability.
- **Diffusion:** random perturbations that allow exploration of the parameter space.

More generally, diffusion models describe stochastic processes driven by drift and diffusion.

What is diffusion?

A process X_t is a diffusion if it has these properties:

- The time variable t is continuous;
- It is a Markov process;
- X_t is a continuous function of t.

A diffusion process X_t can be expressed in Ito form:

$$dX_t = a(X_t)dt + b(X_t)dW_t,$$

where a is a drift, b is a noise coefficient, W_t is a Wiener process.

Diffusion forward process

At each step, Gaussian noise is added to the data:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}), \qquad \alpha_t = 1 - \beta_t.$$

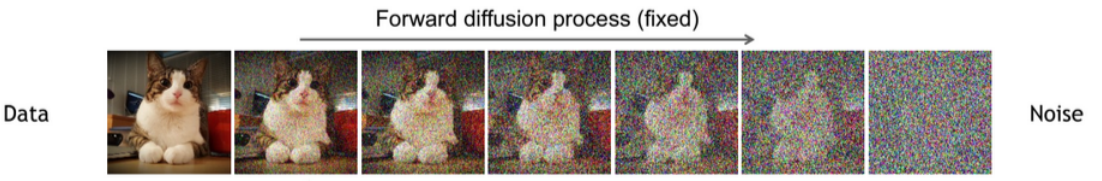
Define:

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

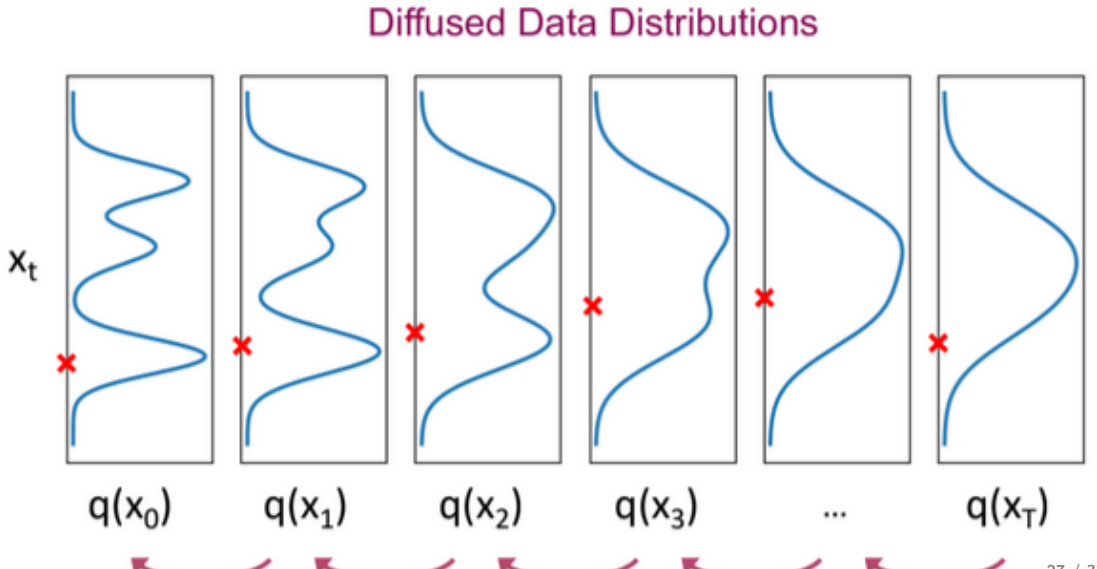
Then the distribution at an arbitrary timestep is:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}).$$

The noise schedule is chosen such that $\bar{\alpha}_T \approx 0$, yielding $q(\mathbf{x}_T|\mathbf{x}_0) \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$.

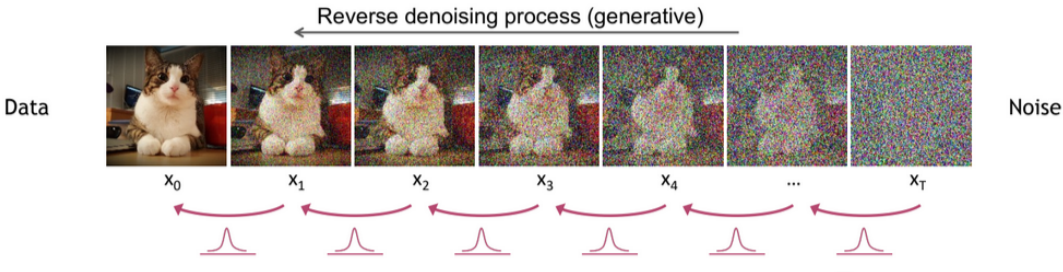


Diffusion forward process



Diffusion backward process

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}(\mathbf{x}_t), \sigma^2\mathbf{I}).$$



Diffusion as a variational model

The ELBO:

$$\begin{aligned}\mathcal{L}_{\text{ELBO}} = &\mathbb{E}_q[\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)] \\ &- \text{KL}(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)) \\ &- \sum_{t=2}^{T} \mathbb{E}_q \text{KL}\left(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\|p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)\right).\end{aligned}$$

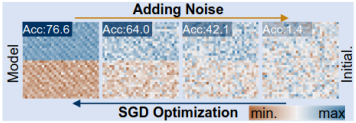
The forward posterior is Gaussian and tractable:

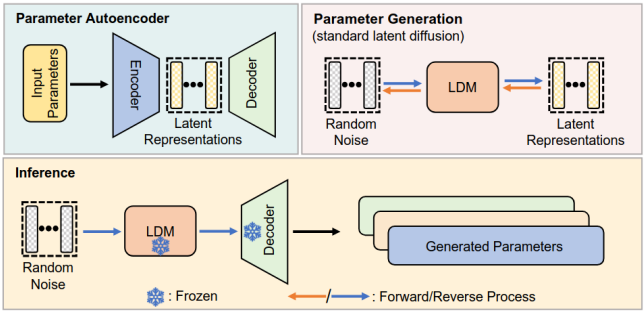
$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\boldsymbol{\mu}}_t, \tilde{\beta}_t \mathbf{I}).$$

The KL term can be computed analytically:

$$\begin{aligned}&\text{KL}\left(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\|p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)\right) \\ &= \text{const} + \mathbb{E}_q \frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon} \right) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t) \right\|^2.\end{aligned}$$

Model parameters generation using diffusion models





Diffusion and score matching

Score matching idea:

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{1}{Z(\boldsymbol{\theta})} \exp\left(-E(\mathbf{x}; \boldsymbol{\theta})\right).$$

To approximate the unknown true distribution $p_{\text{data}}(\mathbf{x})$, we match the scores of the distributions:

$$\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left\| \nabla_{\mathbf{x}} \log p_{\boldsymbol{\theta}}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) \right\|^2 \to \min.$$

The unknown data score can be replaced using denoising score matching:

$$\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}, \hat{\mathbf{x}} \sim q(\hat{\mathbf{x}}|\mathbf{x})} \left\| \nabla_{\hat{\mathbf{x}}} \log p_{\boldsymbol{\theta}}(\hat{\mathbf{x}}) - \nabla_{\hat{\mathbf{x}}} \log q(\hat{\mathbf{x}}|\mathbf{x}) \right\|^2 \to \min,$$

where $q(\hat{\mathbf{x}}) = \int q(\hat{\mathbf{x}}|\mathbf{x}) p_{\text{data}}(\mathbf{x}) d\mathbf{x}$.

Autoencoder: generative model?

(Alain, Bengio 2012): consider regularized autoencoder:

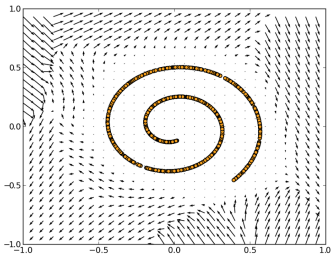
$$||\boldsymbol{f}(\boldsymbol{x}, \sigma) - \boldsymbol{x}||^2,$$

where σ is a noise level.

Then

$$\frac{\partial \log p(\mathbf{x})}{\partial \mathbf{x}} = \frac{||\boldsymbol{f}(\boldsymbol{x}, \sigma) - \boldsymbol{x}||^2}{\sigma^2} + o(1) \text{ when } \sigma \to 0.$$

Vector field induced by reconstruction error



References

- Bishop C. M. Pattern recognition //Machine learning. – 2006. – Т. 128. – №. 9.
- MacKay D. J. C., Mac Kay D. J. C. Information theory, inference and learning algorithms. – Cambridge university press, 2003.
- Salimans, Tim, Diederik Kingma, and Max Welling, 2015. Markov chain monte carlo and variational inference: Bridging the gap
- Altieri: http://approximateinference.org/accepted/AltieriDuvenaud2015.pdf
- Stephan Mandt, Matthew D. Hoffman, David M. Blei, 2017. Stochastic Gradient Descent as Approximate Bayesian Inference
- Бахтеев О. Ю., Стрижов В. В. Выбор моделей глубокого обучения субоптимальной сложности //Автоматика и телемеханика. – 2018. – №. 8. – С. 129-147.
- Hinton, Geoffrey E., and Drew Van Camp. "Keeping the neural networks simple by minimizing the description length of the weights."Proceedings of the sixth annual conference on Computational learning theory. 1993.
- Figurnov M., Mohamed S., Mnih A. Implicit reparameterization gradients //arXiv preprint arXiv:1805.08498. – 2018.
- Jang E., Gu S., Poole B. Categorical reparameterization with gumbel-softmax //arXiv preprint arXiv:1611.01144. – 2016.
- Potapczynski A., Loaiza-Ganem G., Cunningham J. P. Invertible gaussian reparameterization: Revisiting the gumbel-softmax //arXiv preprint arXiv:1912.09588. – 2019.
- Maddison C. J., Mnih A., Teh Y. W. The concrete distribution: A continuous relaxation of discrete random variables //arXiv preprint arXiv:1611.00712. – 2016.
- Shayer O., Levi D., Fetaya E. Learning discrete weights using the local reparameterization trick //arXiv preprint arXiv:1710.07739. – 2017.
- Li Y., Turner R. E. Rényi Divergence Variational Inference //arXiv preprint arXiv:1602.02311. – 2016.
- Hutchinson, M. F. (1990). A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. Communications in Statistics - Simulation and Computation, 19(2), 433–450.
- Rezende, D., Mohamed, S. (2015, June). Variational inference with normalizing flows. In International conference on machine learning (pp. 1530-1538). PMLR.
- Liu, Q., Wang, D. (2016). Stein variational gradient descent: A general purpose bayesian inference algorithm. Advances in neural information processing systems, 29.
- Lectures on Diffusion processes: https://math.nyu.edu/ goodman/teaching/StochCalc2018/notes/Lesson2.pdf
- Tutorial on diffusion models: https://cvpr2023-tutorial-diffusion-models.github.io/
- Wang, Kai, et al. "Neural network diffusion."arXiv preprint arXiv:2402.13144 (2024).
- Vincent, Pascal. "A connection between score matching and denoising autoencoders."Neural computation 23.7 (2011): 1661-1674.
- Alain G., Bengio Y. What regularized auto-encoders learn from the data-generating distribution //The Journal of Machine Learning Research. – 2014. – Т. 15. – №. 1. – С. 3563-3593.